F1 Racing Line Optimization - Full Project Context Report

Date: April 4, 2026 **Repository:** `RL` **Project Type:** Reinforcement Learning for autonomous racing-line discovery

---

1. Executive Summary

This project builds a custom reinforcement learning pipeline for teaching an agent to drive a simplified Formula-style car around a closed circuit and improve its racing behavior over time. The central idea is to learn an effective racing line from interaction, rather than hard-coding one. The system combines:

- a custom `gymnasium.Env`
- a bicycle-model vehicle simulator
- track geometry loaded from TUMFTM-style CSV files
- a 7-ray distance sensor observation model
- Soft Actor-Critic (SAC) from Stable-Baselines3
- curriculum training across multiple stages
- rollout export, visualization, baseline comparison, and cluster support

The project began with a simpler staged plan built around a drag strip and Monza transfer learning, but the current implementation has evolved into a **single-track, three-stage curriculum on `data/tracks/f1.csv`**:

- **Stage 1:** survival / lap completion with checkpoint reward
- **Stage 2:** time-attack optimization on the same track
- **Stage 3:** low-learning-rate fine-tuning on the same track

This shift simplified transfer complexity and made replay-buffer reuse across stages practical.

---

2. Aim of the Project

The project aims to answer a practical RL question:

Can an agent with only local geometric sensing and a simple vehicle model learn to complete a race circuit and improve its racing line without being explicitly given the optimal trajectory?

The broader goals are:

- learn stable closed-loop driving behavior
- move from survival to speed optimization
- study reward shaping and curriculum learning
- compare RL behavior against a classical controller baseline
- create a project that is explainable, visual, and extensible

This is not a full motorsport dynamics simulator. It is a deliberately reduced but meaningful environment for studying sequential decision-making, control, reward design, and training stability.

---

3. Why RL Fits This Problem

Racing line optimization is naturally sequential and delayed-reward in nature:

- a steering decision now affects corner entry, apex, and exit later
- braking too late may look good locally but causes a crash later
- taking a wider or tighter line changes future achievable speed
- the best action depends on state, speed, orientation, and nearby geometry

This makes the problem a good fit for reinforcement learning rather than static supervised prediction.

Core RL framing

- **State / observation:** local perception of track geometry plus current speed
- **Action:** steering and throttle/brake
- **Transition:** vehicle physics + track geometry
- **Reward:** survival, progress, and later speed-weighted progress
- **Policy objective:** maximize long-term return

---

4. Main Approach

The overall approach is:

1. Build a custom racing simulator with enough realism to make the control problem meaningful.
2. Keep the observation compact so the agent must infer behavior from local geometry.
3. Use curriculum learning so the agent first learns to survive before learning to go fast.
4. Use SAC because it is robust on continuous-control problems with bounded actions.

5. Add strong diagnostics: manual driving, rollouts, trajectory CSVs, visualizations, checkpoint evolution plots, and a PID baseline.

This approach reflects a practical engineering philosophy: start with a learnable problem, remove obvious local optima, and only then optimize behavior.

---

5. Theory and Algorithms Used

5.1 Reinforcement Learning

The agent interacts with the environment repeatedly:

- observes a state
- picks an action
- receives reward
- transitions to a new state

Over many episodes, it learns a policy that improves expected cumulative reward.

The project uses **off-policy actor-critic learning**, which is suitable for:

- continuous steering/throttle spaces
- replay buffer reuse
- sample efficiency
- staged fine-tuning from earlier checkpoints

5.2 Soft Actor-Critic (SAC)

The implementation uses `stable_baselines3.SAC` with `MlpPolicy`.

Why SAC is a good choice here:

- supports continuous actions directly
- learns a stochastic policy, which improves exploration
- uses entropy regularization to avoid collapsing too early
- works well for robotics/control-like tasks
- can reuse replay buffers when environment semantics remain compatible

In this project, SAC is used with:

- multilayer perceptron policy
- replay buffer of size `1_000_000`
- batch size `256`
- evaluation callbacks
- checkpointing during training

5.3 Curriculum Learning

Curriculum learning is one of the most important design decisions in the repository.

Instead of asking the agent to discover both survival and speed optimization at once, the training is decomposed:

- first learn not to crash
- then learn to move efficiently
- then refine behavior conservatively

This reduces exploration difficulty and improves stability.

5.4 Classical Control Baseline

The repo includes a simple **PID-style baseline** controller in `baseline.py`.

Its role is not to be a perfect racing policy. It serves as:

- a sanity-check that the environment is drivable
- a classical comparison point
- a simple centerline follower against which RL behavior can be judged

5.5 Geometric Track Modeling

Track data is loaded from CSV with:

- centerline points
- left/right widths

The centerline is smoothed using a periodic cubic spline and resampled. Track bounds are reconstructed from centerline normals and widths. This gives:

- smoother geometry than raw waypoint chains
- consistent sensor intersection behavior
- better rendering and progress computation

5.6 Bicycle Kinematic Vehicle Model

The car is modeled with a simplified bicycle model:

- position `(x, y)`
- heading
- scalar speed
- steering input
- acceleration/braking input

The model adds:

- max steer clamp
- max acceleration and braking
- rolling friction
- aerodynamic drag
- turning-induced speed scrub
- cornering grip limit through maximum lateral acceleration

This is much simpler than full tire dynamics, but it is appropriate for an RL prototype.

---

6. Current Architecture

Core files

- `train.py` : curriculum training entrypoint
- `config.py` : project constants
- `env/race.py` : Gymnasium environment
- `env/car.py` : vehicle model and ray sensors
- `env/track.py` : CSV loading, smoothing, boundaries, progress logic
- `env/reward.py` : reward modes
- `rollout.py` : deterministic policy playback and CSV export
- `visualize.py` : racing-line and speed-profile plots
- `baseline.py` : PID centerline-following baseline
- `checkpoint_viz.py` : checkpoint-by-checkpoint racing-line evolution
- `manual_mode.py` : manual driving and visual debugging
- `track_editor.py` : interactive creation/editing of track CSV files
- `slurm/train_stage.sbatch` : cluster training job
- `CLUSTER_GUIDE.md` : cluster workflow documentation

Data and outputs

- `data/tracks/` : track CSV files
- `outputs/trajectories/` : rollout trajectory CSVs
- `outputs/plots/` : generated figures
- `models/` : intended staged model storage
- `logs/` : TensorBoard / evaluation logs

---

7. Environment Design

7.1 Observation Space

The environment returns an `(N_SENSORS + 1,)` vector, currently:

- 7 ray distances
- 1 normalized speed scalar

The ray angles are:

- `[-60, -40, -20, 0, 20, 40, 60]` degrees

Distances are normalized to `[0, 1]` in the car model and then mapped to `[-1, 1]` in the environment. Speed is similarly normalized and mapped to `[-1, 1]`.

Interpretation

This makes the problem partly reactive:

- the agent sees free space around its heading
- it does not directly observe a global map
- it must infer how to brake, turn, and accelerate from local geometry

This compact state is elegant, but it also limits long-horizon anticipation on complex corners.

7.2 Action Space

The action is a 2D continuous vector:

- steering in `[-1, 1]`
- acceleration in `[-1, 1]`

The second component is interpreted as:

- positive = throttle
- negative = brake

7.3 Episode Logic

An episode ends if:

- the car leaves the track
- anti-stall termination triggers
- the max step limit is reached

Current settings in `env/race.py`:

- `MAX_STEPS = 5000`
- anti-stall if no new waypoint is reached for `>100` steps
- anti-stall adds an extra `-5.0` penalty

This anti-stall rule is one of the project's most important practical fixes.

---

8. Vehicle Dynamics and Physics Model

The car implementation in `env/car.py` includes the following key constants from `config.py`:

- `WHEELBASE = 2.5`
- `MAX_STEER = 0.8`
- `MAX_ACCEL = 15.0`
- `MAX_BRAKE = 15.0`
- `MAX_SPEED = 60.0 m/s`
- `FRICTION = 5.5`
- `DRAG = 0.001`
- `CORNERING_G = 2.0`
- `TURN_FRICTION = 10.0`
- `DT = 0.05`

Important design choices

Rolling start

Each reset gives the car an initial speed of `10.0 m/s`.

Reason:

- starting from rest made forward exploration too difficult
- random early policies often failed to discover useful momentum
- the rolling start produces meaningful trajectories immediately

Grip-limited steering

At higher speeds, the effective steering angle is reduced based on a lateral-acceleration cap. This prevents unrealistic instant turning and makes corner entry planning matter.

Turn friction

Turning scrubs speed, which encourages realistic trade-offs:

- too much steering at high speed loses momentum
- smooth lines become more valuable than jerky corrections

Overall, the physics model is simple but purposely shaped to support meaningful racing behavior.

---

9. Track Representation and Geometry

`env/track.py` is one of the most important modules in the repository.

Responsibilities

- read TUMFTM-style CSV geometry
- remove duplicate closing point if present
- fit a periodic cubic spline
- resample to a smoother centerline
- interpolate widths
- compute left/right boundaries
- compute cumulative distances
- answer nearest waypoint / progress / on-track queries

Why smoothing matters

Without smoothing:

- raw points can create jagged geometry
- sensor intersections become noisy
- progress estimates become unstable
- rendering looks rough

On-track detection

The project moved away from naive nearest-waypoint logic and now uses **point-to-segment distance** with width interpolation.

This is a strong fix because nearest-waypoint checks can fail on geometries where nearby parts of the track are spatially close but topologically different. The current implementation is much more robust.

Important caveat

`Track.progress()` currently returns **cumulative distance** at the nearest waypoint, not normalized lap progress in `[0, 1]`.

Meanwhile, `env/reward.py` includes wrap-around logic that assumes normalized progress:

- it checks whether `progress_delta < -0.5`
- it then uses `(1.0 - prev_progress) + curr_progress`

That logic conceptually belongs to normalized progress, not absolute meters. This is a known conceptual mismatch in the current codebase and should be documented as a future cleanup target.

---

10. Reward Design

Reward design has clearly been a central theme of the project.

Two modes are implemented in `env/reward.py`.

10.1 Checkpoint Reward

Used in Stage 1.

Form:

- constant step penalty
- reward for new waypoints reached
- off-track penalty on crash

Purpose:

- teach movement and lap completion
- avoid the early sparse-reward failure mode
- establish stable driving before speed optimization

10.2 Lap-Time Reward

Used in Stages 2 and 3.

Form:

- constant step penalty
- reward proportional to progress change times speed
- off-track penalty on crash

Purpose:

- make faster forward motion more valuable
- shift the objective from survival to efficient lap completion

Why this staged design is good

If speed reward is introduced too early:

- the agent may drive aggressively before understanding corner safety
- training can collapse into frequent crashing

By learning survival first, later speed optimization starts from a competent driving prior.

---

11. Training Pipeline

The active curriculum defined in `train.py` is:

Stage	Name	Track	Reward	Steps	LR
1	Survival - Custom F1	`data/tracks/f1.csv`	`checkpoint`	400,000	`3e-4`
2	Time Attack - Custom F1	`data/tracks/f1.csv`	`laptime`	500,000	`1e-4`
3	Fine Tune - Custom F1	`data/tracks/f1.csv`	`laptime`	300,000	`5e-5`

Stage transition behavior

- Stage 2 loads Stage 1 weights if available
- Stage 3 loads Stage 2 weights if available
- replay buffer is reused when the current stage track equals the previous stage track

Because all current stages use `f1.csv`, replay buffer reuse is enabled across the full curriculum.

Environment parallelism

`train.py` supports:

- `DummyVecEnv` for single-env training
- `SubprocVecEnv` for multi-env cluster training

Evaluation and checkpoint frequency are scaled by `n_envs` to keep wall-clock cadence more comparable.

Callbacks

- `EvalCallback` for periodic evaluation and best-model saving
- `CheckpointCallback` for intermediate checkpoint files

Training command examples

```
python train.py --stage 1
python train.py --stage 2
python train.py --stage 3
python train.py --stage 1 --steps 100000
python train.py --stage 1 --headless --n-envs 8
```

---

12. Tooling Around Training

12.1 Rollouts

`rollout.py` loads a trained model and runs deterministic episodes. It can:

- render the policy
- print per-step actions
- accelerate playback using multiple physics steps per frame
- export trajectory CSVs

This is useful for:

- verifying that reward improvements correspond to better driving
- diagnosing crashes or hesitation
- generating downstream plots

12.2 Visualization

`visualize.py` plots:

- the learned racing line as a speed heatmap
- a speed profile over time

This is a strong deliverable for a final report because it makes the learned behavior visible.

12.3 Checkpoint Evolution

`checkpoint_viz.py` is especially valuable because it visualizes policy evolution across training checkpoints. This supports a richer narrative than only showing the final model.

12.4 Manual Driving

`manual_mode.py` lets a human drive the environment, which is useful for:

- verifying track correctness
- testing physics feel
- checking camera/render behavior
- validating that the task is actually solvable

12.5 Track Editing

`track_editor.py` provides an interactive spline-based editor for custom track creation and modification. This is a notable project strength because it turns the repo into both an RL system and a content-authoring workflow.

---

13. Rendering and Presentation Layer

The rendering system in `env/race.py` is more polished than a typical debug-only RL environment.

Features include:

- track asphalt and kerb rendering
- centerline dashes
- sensor ray visualization
- car sprite rendering using `assets/mercedes.png`
- HUD with speed, lap progress, step count, and on/off-track status
- follow-camera or full-track camera modes
- visual smoothing of heading and ray endpoints

This matters because visual fidelity improves:

- debugging quality
- demo value
- report screenshots
- intuition about what the agent is doing

The code comments mention supersampling anti-aliasing, but `_SSAA` is currently set to `1`, so supersampling is effectively disabled in the present version.

---

14. Project Evolution

The older reports in `reports/` tell an important story:

- the project originally focused on `drag_strip.csv`
- then planned transfer to `monza.csv`
- several training pathologies were discovered and fixed
- later, the codebase moved to a custom `f1.csv` track for all stages

This evolution makes sense. The current version prioritizes:

- curriculum stability
- simpler stage handoffs
- replay buffer compatibility
- reduced confounding from cross-track transfer

In other words, the project became more coherent as an engineering system, even if that meant narrowing scope compared to the original multi-track transfer ambition.

---

15. Major Mistakes, Problems, and Lessons Learned

This is one of the strongest parts of the project. Several classic RL/control failure modes were encountered and addressed.

15.1 The agent preferred standing still

Problem

If crashing produced a large negative reward and doing nothing was weakly punished, the agent could learn that inaction was safer than exploration.

Solution

- add a per-step penalty
- add anti-stall termination
- use a rolling start

Lesson

In RL, agents optimize the objective literally, not the intended behavior. Reward shaping must explicitly reject degenerate local optima.

15.2 Standing-start exploration was too hard

Problem

With zero initial speed and imperfect early actions, the policy struggled to discover useful forward motion.

Solution

- initialize the car at `10.0 m/s`
- increase acceleration authority

Lesson

Sometimes the main issue is not the algorithm but the exploration geometry of the task.

15.3 Window freezing / poor interactivity

Problem

Pygame windows can freeze if the event queue is not serviced.

Solution

- pump the event queue during render

Lesson

A usable RL environment is also a software product. Tooling and interactivity matter.

15.4 On-track detection was initially too naive

Problem

Nearest-waypoint logic can produce false off-track decisions on geometries with nearby parallel segments.

Solution

- switch to point-to-segment distance with interpolated widths

Lesson

Geometry shortcuts become training bugs when reward and termination depend on them.

15.5 The original plan was broader than the stabilized implementation

Problem

The older reports emphasize drag-strip survival followed by Monza transfer, but the current code has consolidated into a same-track curriculum.

Solution

- simplify the curriculum to one custom F1 track
- preserve the staged learning idea

Lesson

Reducing scope can be a strength when it improves experimental control and implementation reliability.

---

16. Current Status of the Project

Based on the repository contents as of April 4, 2026:

- the core environment and training pipeline are implemented
- cluster workflow has been prepared
- trajectory and plot outputs exist
- checkpoint visualization support exists
- stage-based SAC training is operational in code
- multiple prior reports document project progress and planning

There is also evidence of local training artifacts in the repository root:

- `best_model.zip`
- `final_model.zip`
- `stage1_sac_88000_steps.zip`
- `stage1_sac_144000_steps.zip`

These do not match the scripted save layout in `train.py`, which expects staged outputs under `models/stageN/`. This suggests that some training artifacts were copied or exported manually during experimentation.

Practical interpretation

The project is well beyond prototype stage, but not fully cleaned up as a final, publication-style research codebase. It is in a strong "working project with real results and a few consistency issues" state.

---

17. Known Inconsistencies and Technical Debt

For a full-context report, these should be recorded honestly.

17.1 Old reports vs current code

Older reports describe:

- drag strip curriculum
- Monza transfer
- stage semantics that no longer match current `train.py`

Current code uses `f1.csv` across all stages.

17.2 Progress semantics mismatch

As noted earlier:

- `Track.progress()` returns distance in meters
- `reward.py` includes wrap-around logic that appears written for normalized progress

This should be aligned.

17.3 Visualization column mismatch

`rollout.py` exports `speed_kmh`, but `visualize.py`'s `plot_speed_profile()` currently reads `df["speed"]`.

That means speed-profile plotting is likely inconsistent or broken for rollout-generated CSVs unless a different CSV format is used.

17.4 Save-path inconsistency

The intended model layout is stage-specific under `models/`, but several model files currently sit at repository root.

17.5 Config as partial source of truth

`config.py` contains defaults, but `train.py` mutates `TRACK_FILE`, `REWARD_MODE`, and `HEADLESS` at runtime per stage. This is practical, but it means:

- config is not the sole source of truth
- script behavior depends on call order and entrypoint usage

---

18. Scientific / Technical Interpretation

What this project is really demonstrating is not just "RL can drive a car."

It demonstrates several deeper points:

RL performance is highly dependent on environment design

The biggest breakthroughs in the repo came from:

- reward shaping
- reset design
- geometry correctness
- termination rules

not from changing the algorithm.

Curriculum matters

Breaking the task into survival and optimization is more effective than asking one policy to solve everything at once.

Perception limits behavior

A 7-ray observation is elegant and compact, but it likely limits:

- long-horizon anticipation
- handling of compound corners
- sophisticated line planning on very complex tracks

A good baseline is essential

Including a PID baseline is a very good methodological choice. It prevents overclaiming and gives a classical control point of comparison.

---

19. What Counts as the "Solution"

The project's solution is not only the trained model. It is the full system:

- a custom racing environment
- a shaped reward design
- a curriculum training strategy
- an off-policy continuous-control learner
- supporting analysis tools
- a comparison baseline
- a cluster-ready training workflow

The most important solved problems are:

- getting the agent to move meaningfully
- preventing stall-based local optima
- making track membership robust
- transitioning from survival to speed optimization
- building a workflow to inspect and explain learned behavior

---

20. Strengths of the Project

- Clear problem formulation with a meaningful RL objective
- Well-chosen algorithm for continuous control
- Strong environment engineering for a student/project-scale simulator
- Honest and useful reward/termination shaping
- Good supporting tooling for visualization and analysis
- Cluster-ready training setup
- Good educational value: the code exposes the full RL pipeline end to end

---

21. Limitations

- observation is local and low-dimensional
- no explicit racing line target or expert imitation
- no full dynamic tire model
- current curriculum no longer tests true cross-track transfer
- code/docs have some drift
- evaluation reporting is not yet consolidated into one reproducible experiment table

These are reasonable limitations for the scope.

---

22. Recommended Next Steps

If this project is being prepared for final submission, demo, or further development, the highest-value next steps are:

1. **Unify documentation with the current code.** Replace older drag-strip/Monza status assumptions with the current `f1.csv` curriculum.
2. **Fix progress semantics.** Decide whether progress should be normalized `[0, 1]` or absolute meters, then align `track.py` and `reward.py`.
3. **Fix visualization CSV compatibility.** Make `visualize.py` consistently use `speed_kmh` or update rollout export accordingly.
4. **Consolidate model artifacts.** Move experimental root-level model files into the stage directory structure.
5. **Produce a final evaluation bundle.** Include:
    - final trajectories
    - PID baseline trajectories
    - heatmap plot
    - checkpoint evolution plot
    - lap-time / completion-rate comparison table
6. **Optionally restore transfer learning as a later extension.** Once the single-track pipeline is stable and documented, add a separate transfer experiment such as:
    - `f1.csv -> monza.csv`
    - `drag_strip -> test_oval -> monza`

---

23. Final Conclusion

This project is a strong reinforcement learning systems project centered on autonomous racing-line optimization. Its main contribution is not merely a trained driver, but the engineering of a full learning environment in which the agent can gradually acquire useful racing behavior.

The project shows a clear maturation path:

- from broad idea
- to environment implementation
- to RL failure diagnosis
- to reward and termination redesign
- to curriculum stabilization
- to analysis tooling and cluster scalability

Its most important lesson is that successful RL work depends as much on **problem framing and environment design** as on the learning algorithm itself. The repo now contains a coherent, explainable, and extensible platform for studying autonomous racing behavior with reinforcement learning.

Appendix A - Historical Narrative

The repository history, reports, and file set suggest the following narrative:

- The project started with a general F1 racing-line optimization goal.
- Early work used a drag-strip style curriculum to simplify learning.
- A future transfer to Monza was planned.
- Several reward and environment pathologies were discovered.
- Important fixes were introduced: step penalty, anti-stall, rolling start, stronger acceleration/braking, and improved on-track logic.
- The codebase later consolidated onto a custom `f1.csv` track for all stages, likely to reduce instability and make training more reproducible.
- Additional tooling was added for rollouts, visualization, checkpoint evolution, track editing, and cluster execution.

This is a healthy project evolution: simplify where needed, then strengthen infrastructure.

Appendix B - Key Commands

```
# Train
python train.py --stage 1
python train.py --stage 2
python train.py --stage 3

# Cluster / headless
python train.py --stage 1 --headless --n-envs 8

# Rollout
python rollout.py --model models/stage1/best_model --episodes 3

# Baseline
python baseline.py

# Visualize
python visualize.py --trajectory outputs/trajectories/trajectory_ep1.csv

# Checkpoint evolution
python checkpoint_viz.py --stage 1

# Track editing
python track_editor.py --file data/tracks/custom_track.csv
```

Appendix C - Referenced Existing Reports

The following existing documents informed this full-context report:

- `reports/project_status2.md`
- `reports/PROGRESS1.md`
- `reports/FUTURE_PLAN.md`

Where those documents differ from the current code, this report prioritizes the present implementation while preserving the historical context.